MISSING NUMBER IN AN ARRAY

Name	**:**	Rudra Pratap Singh
SAP ID	**:**	590032811
Batch	**:**	16

1. Problem Statement

Write a C program that reads the size and the elements of a variable-length integer array nums[]. The array contains distinct integers taken from the range 0 to size, with exactly one number missing. The array must be passed to a function named missing_number() using its address, and pointers must be used to access it. The function must find and display the missing integer.

2. Aim

To find the one missing number from the range 0 to size in a given array of distinct integers, using pointers, dynamic memory allocation and a user-defined function in C.

3. Concepts Used

- Pointers and pointer arithmetic (*ptr, ptr++, nums + i)
- Passing an array to a function by its address
- Dynamic memory allocation using malloc() and free()
- Looping, user-defined functions and input/output using scanf() and printf()

4. Logic and Approach

The array holds size numbers chosen from size + 1 possible values (0, 1, 2, ..., size). Since all numbers are distinct and only one is absent, the missing number can be found with a simple sum formula.

The sum of all integers from 0 to size is:

$$\textbf{Expected Sum} = \textbf{size} \times (\textbf{size} + 1) / 2$$

The sum of the elements actually present in the array is calculated using a pointer. The difference between the two sums is the missing number:

$$\textbf{Missing Number} = \textbf{Expected Sum} - \textbf{Actual Sum}$$

Example: For size = 3 and array = {3, 0, 1}, the expected sum is $3 \times 4 / 2 = 6$ and the actual sum is $3 + 0 + 1 = 4$. Therefore the missing number is $6 - 4 = 2$.

5. Algorithm

1. Start.
2. Read the value of size from the user.
3. Allocate memory for size integers using malloc() and read the elements into nums[].
4. Call missing_number(nums, size), passing the address of the array.
5. Inside the function, compute expected_sum = size × (size + 1) / 2.

6. Using a pointer, traverse the array and add every element to actual_sum.
7. Compute missing = expected_sum − actual_sum.
8. Display the missing number.
9. Free the allocated memory and stop.

6. Source Code

```c
/*
 * Program : Missing Number in an Array
 * Name    : Rudra Pratap Singh
 * SAP ID  : 590032811
 * Batch   : 16
 *
 * Description:
 * The array nums[] holds 'size' distinct integers taken from the
 * range 0 to size, so exactly one number of that range is missing.
 * The missing number is found using the sum formula:
 *         missing = size*(size+1)/2 - (sum of array elements)
 */

#include <stdio.h>
#include <stdlib.h>

/* Finds and displays the missing number. Array is accessed via pointer. */
void missing number(int *nums, int size)
{
    long expected_sum = (long)size * (size + 1) / 2;  /* sum of 0..size */
    long actual sum = 0;
    int *ptr;

    for (ptr = nums; ptr < nums + size; ptr++)
    {
        actual sum += *ptr;
    }

    printf("Missing number: %d\n", (int)(expected_sum - actual_sum));
}

int main(void)
{
    int size, i;
    int *nums;

    printf("Size: ");
    if (scanf("%d", &size) != 1 || size < 1)
    {
        printf("Invalid size. Please enter a positive integer.\n");
        return 1;
    }

    /* Allocate memory for a variable-length array */
    nums = (int *)malloc(size * sizeof(int));
    if (nums == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Array: ");
    for (i = 0; i < size; i++)
    {
        scanf("%d", nums + i);
    }

    missing_number(nums, size);   /* pass address of the array */

    free(nums);
    return 0;
}
```

7. Sample Outputs

The program was compiled with gcc using the -Wall -Wextra options (no warnings) and tested on all the given examples.

Test Case 1

```
Size: 3
Array: 3 0 1
Missing number: 2
```

Test Case 2

```
Size: 2
Array: 0 1
Missing number: 2
```

Test Case 3

```
Size: 9
Array: 9 6 4 2 3 5 7 0 1
Missing number: 8
```

Test Case 4

```
Size: 1
Array: 0
Missing number: 1
```

8. Complexity Analysis

Measure	Complexity	Reason
Time Complexity	O(n)	Array is traversed once
Space Complexity	O(1)	Only a few variables are used in the function

9. Conclusion

The program successfully finds the missing number from the range 0 to size using the sum formula. The array was passed to missing_number() by its address and accessed entirely through pointers. The solution is efficient, as it needs only a single pass over the array, and it produces the correct result for all the given test cases.